Program Studi Teknik Informatika
Institut Teknologi Sumatera

IF25-22014 PENGEMBANGAN APLIKASI WEB

Modul Belajar 11

Kerangka Kerja FastAPI

Membangun API JSON yang divalidasi otomatis dan dipanggil dari Next.js

Tim Pengajar
Pengembangan Aplikasi Web. Tahun 2026

Komponen	Keterangan
Pertemuan	11 dari 16
Capaian	Sub-CPMK 11.1 dan 11.2, turunan CPMK0502
Waktu belajar	sekitar 3 jam belajar mandiri, ditambah 150 menit di kelas
Hands-on	`02 Hands-on/P11 - FastAPI - Hands-on/`
Yang perlu disiapkan	Python 3.11+ dengan venv, `pip install "fastapi[standard]"`, dan proyek Next.js dari P7

Peta belajar

Modul ini mengikuti bagian yang sama dengan slide. Setiap konsep inti ditulis dalam kotak gelap: itulah kalimat yang harus bisa kalian jelaskan ulang.

Bagian	Judul	Yang dipelajari
1	Pengenalan FastAPI	Menjelaskan FastAPI dan alasan memilihnya untuk API
2	Arsitektur FastAPI	Menjelaskan perjalanan request di dalam aplikasi FastAPI
3	Instalasi dan konfigurasi	Memasang, menjalankan, dan mengonfigurasi FastAPI termasuk CORS
4	Routing	Membuat path operation dengan path parameter, query parameter, dan APIRouter
5	Request dan response model	Memvalidasi body dengan model Pydantic dan mengatur response
6	Model dan integrasi basis data	Memisahkan model API dari penyimpanan dan menyiapkan dependency basis data
7	Keamanan dan autentikasi	Menjelaskan autentikasi dengan token dan praktik keamanan dasar API
8	API dan RESTful services	Merancang endpoint mengikuti prinsip REST dan memanggilnya dari Next.js

Tentang modul ini

Modul ini menemani Pertemuan 11. Setiap kotak merah **Tebak dulu** meminta kalian menulis dugaan sebelum membaca kotak hijau **Jawaban** di bawahnya. Kerjakan contoh kode sambil membaca, jangan hanya dibaca.

Setelah menyelesaikan modul ini, kalian bisa

No	Kemampuan	Diukur di
11.1	Menjelaskan arsitektur FastAPI: routing, validasi Pydantic, dependency, dan dokumentasi otomatis	Exit ticket soal 1, hands-on, proyek
11.2	Membangun RESTful API dengan path, query, body, kode status, dan CORS untuk Next.js	Exit ticket soal 2, hands-on, proyek

Keduanya menurunkan CPMK0502: mahasiswa mampu menjelaskan konsep pemrograman untuk pengembangan perangkat lunak.

Masalah pembuka

Halaman siap, datanya dari mana?

Halaman KRS Next.js dari P7 sudah memanggil `http://localhost:8000/matkul`, tetapi alamat itu belum ada: browser menampilkan *Gagal memuat*.

Data mata kuliah juga harus diperiksa sebelum disimpan, karena validasi di browser bisa dilewati (P1).

> **TEBAK DULU**
> Program seperti apa yang harus berjalan di port 8000? Apa yang ia terima, dan apa yang ia kirim balik?

```
useEffect(() => {
  fetch("http://localhost:8000/matkul")
    .then((r) => r.json())
    .then(setData)
    .catch(setGalat);
}, []);
// TypeError: Failed to fetch
```

Bagian 1: Pengenalan FastAPI

TUJUAN: MENJELASKAN FASTAPI DAN ALASAN MEMILIHNYA UNTUK API

Kerangka kerja API Python

> FastAPI
> Kerangka kerja web Python untuk membangun API yang memakai type hints untuk memvalidasi data masuk, mengubah data keluar menjadi JSON, dan membuat dokumentasi interaktif secara otomatis.

Pyramid di versi lama mata kuliah ini menyajikan halaman HTML dari server. Di stack baru, halaman dibangun Next.js; server cukup menyediakan data JSON. FastAPI dibuat khusus untuk itu, cepat karena berbasis ASGI dan mendukung `async`.

```
from fastapi import FastAPI

app = FastAPI(title="API KRS")

@app.get("/")
def beranda():
    return {"pesan": "API KRS berjalan"}
```

Inti Bagian 1

FastAPI membaca type hints untuk validasi, JSON, dan dokumentasi otomatis.

Bagian 2: Arsitektur FastAPI

TUJUAN: MENJELASKAN PERJALANAN REQUEST DI DALAM APLIKASI FASTAPI

Perjalanan satu request

Pyramid memakai pola MVC dengan template. FastAPI berperan sebagai lapisan API saja:

1 Masuk

Middleware (misalnya CORS) memeriksa request, lalu path dan metode dicocokkan ke **path operation**.

2 Diperiksa

Parameter dan body divalidasi **model Pydantic**; dependency seperti sesi basis data disiapkan.

3 Dibalas

Fungsi mengembalikan object Python; FastAPI menyaring dengan **response model** lalu mengirim JSON.

Di proyek: `main.py` membuat app, `routers/` berisi path operation per sumber daya, `schemas.py` model Pydantic, `models.py` tabel basis data (P12).

Inti Bagian 2

Middleware, path operation, validasi Pydantic, dependency, lalu balasan JSON.

Bagian 3: Instalasi dan konfigurasi

TUJUAN: MEMASANG, MENJALANKAN, DAN MENGONFIGURASI FASTAPI TERMASUK CORS

Pasang dan jalankan

```
python -m venv .venv
source .venv/bin/activate # Windows:  .venv\Scripts\activate
pip install "fastapi[standard]"

fastapi dev main.py
# Serving at:  http://127.0.0.1:8000
# API docs:  http://127.0.0.1:8000/docs

# alternatif
uvicorn main:app --reload --port 8000
```

`fastapi[standard]` memasang FastAPI beserta server Uvicorn dan alat baris perintah.

`fastapi dev` menjalankan server pengembangan yang memuat ulang otomatis saat kode disimpan.

`/docs`: dokumentasi interaktif Swagger UI dibuat otomatis dari kode. Jawaban misi 1.

`uvicorn main:app`: menjalankan object `app` di berkas `main.py`; dipakai juga saat deployment.

CORS untuk Next.js

```
from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:3000"],
    allow_methods=["*"],
    allow_headers=["*"],
)
```

Next.js di port 3000 dan FastAPI di port 8000 dianggap browser sebagai **asal berbeda**.

Tanpa izin, browser memblokir response dengan pesan CORS di Console, walaupun server sudah membalas 200 di tab Network.

`allow_origins`: daftar asal yang diizinkan. Jangan memakai `"*"` di produksi bila memakai cookie atau token.

Inilah salah satu penyebab *Failed to fetch* di masalah minggu ini.

Ikuti bersama: server FastAPI pertama

Kerjakan sambil membaca, satu langkah demi satu langkah.

No	Langkah	Perintah atau kode
1	Aktifkan venv, lalu pasang FastAPI.	`pip install "fastapi[standard]"`
2	Buat `main.py` berisi app dan endpoint `/` dari Bagian 1.	
3	Jalankan server pengembangan.	`fastapi dev main.py`
4	Buka dokumentasinya.	`http://localhost:8000/docs`
5	Pilih endpoint GET, klik *Try it out*, lalu *Execute*.	

YANG HARUS TERLIHAT

Terminal menulis alamat server dan docs. Halaman /docs menampilkan endpoint kalian, dan Execute memberi respons 200 berisi JSON.

Kalau berbeda

fastapi: command not found: venv belum aktif. *Address already in use*: ada server lain di port 8000; hentikan atau tambah `--port 8001`. Perubahan tidak terlihat: simpan berkasnya.

Inti Bagian 3

Pasang di venv, jalankan fastapi dev, izinkan asal Next.js lewat CORS.

Bagian 4: Routing

TUJUAN: MEMBUAT PATH OPERATION DENGAN PATH PARAMETER, QUERY PARAMETER, DAN APIROUTER

Path dan query parameter

```
@app.get("/matkul")
def daftar(sks: int | None = None, batas: int = 10):
    hasil = [m for m in DATA if sks is None or m["sks"] == sks]
    return hasil[:batas]

@app.get("/matkul/{kode}")
def detail(kode: str):
    for m in DATA:
        if m["kode"] == kode:
            return m
    raise HTTPException(status_code=404, detail="Mata kuliah tidak ditemukan")

@app.get("/halo/{nama}")
def halo(nama: str):
    return {"pesan": f"Halo, {nama}"}
```

`@app.get("/matkul")`: metode GET dan path, setara route di P1.

Query parameter: parameter fungsi yang bukan bagian path, misalnya `/matkul?sks=3&batas=5`. Type hint `int` membuat FastAPI mengubah teks menjadi angka dan menolak bila bukan angka.

Path parameter `{kode}`: bagian URL yang menunjuk satu sumber daya.

`HTTPException(404)`: kode status yang tepat, bukan 200 dengan pesan. Ingat engsel P1.

Jawaban misi 2. Untuk banyak sumber daya, kelompokkan dengan `APIRouter(prefix="/matkul")`.

Inti Bagian 4

Dekorator membuat endpoint; path parameter menunjuk, query parameter menyaring.

Bagian 5: Request dan response model

TUJUAN: MEMVALIDASI BODY DENGAN MODEL PYDANTIC DAN MENGATUR RESPONSE

Model untuk masuk dan keluar

```
from pydantic import BaseModel, Field

class MatkulIn(BaseModel):
    kode: str = Field(pattern=r"^IF\d{2}-\d{5}$")
    nama: str = Field(min_length=3)
    sks: int = Field(ge=1, le=6)

class MatkulOut(MatkulIn):
    id: int

@app.post("/matkul", response_model=MatkulOut, status_code=201)
def tambah(m: MatkulIn):
    baru = {"id": len(DATA) + 1, **m.model_dump()}
    DATA.append(baru)
    return baru
```

Templating di Pyramid diganti model data: tampilan urusan Next.js.

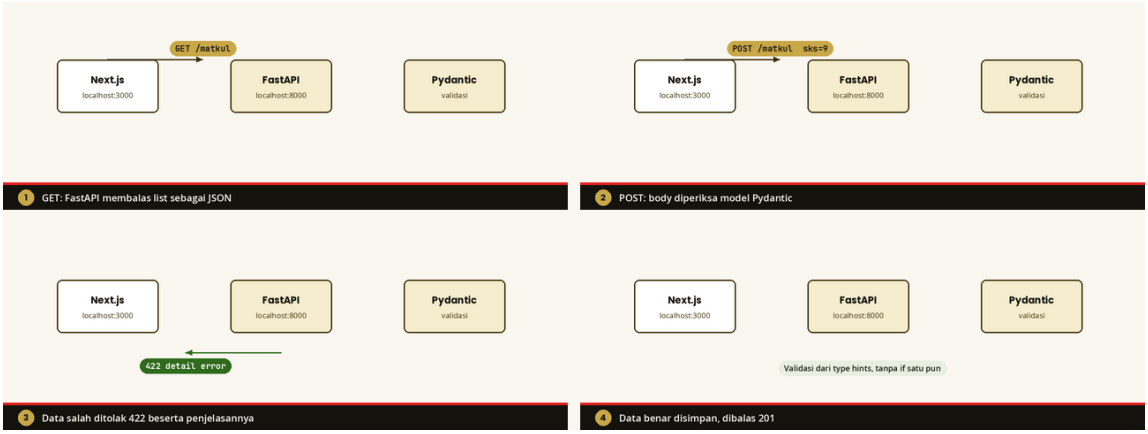
`MatkulIn`: bentuk data yang diterima. Aturan ditulis dengan `Field`: pola kode, panjang nama, rentang SKS.

Parameter bertipe model dibaca dari **body** JSON. Data salah ditolak **422** dengan detail kolom yang salah, sebelum fungsi dijalankan. Jawaban misi 3.

`response_model`: hanya kolom di `MatkulOut` yang dikirim balik, jadi data rahasia tidak bocor.

`status_code=201`: Created, kode yang tepat untuk data baru (P1).

Validasi dalam gerak



Satu endpoint menolak data salah dan menerima data benar tanpa satu pun if di kode kalian. Versi bergeraknya ada di slide PowerPoint.

Kode status yang tepat

TEBAK DULU

Endpoint `PUT /matkul/{kode}` dipanggil untuk kode yang tidak ada. Balasan yang paling tepat? (A) 200 dengan `null`; (B) 404 Not Found; (C) 422 Unprocessable; (D) 500 Server Error

JAWABAN: B

Sumber daya yang dituju tidak ada: 404. 422 untuk body yang tidak lolos validasi, 500 untuk kesalahan kode server.

Inti Bagian 5

Body divalidasi model Pydantic, ditolak 422 bila salah; response model menyaring keluaran.

Bagian 6: Model dan integrasi basis data

TUJUAN: MEMISAHKAN MODEL API DARI PENYIMPANAN DAN MENYIAPKAN DEPENDENCY BASIS DATA

Dependency injection

```
from typing import Annotated
from fastapi import Depends

def get_db():
    db = SesiBasisData() # dibuat per request
    try:
        yield db
    finally:
        db.close()

DB = Annotated[Sesi, Depends(get_db)]

@app.get("/matkul")
def daftar(db: DB):
    return db.ambil_semua_matkul()
```

Sampai hari ini data disimpan di list Python dan hilang saat server dimatikan. P12 menggantinya dengan PostgreSQL.

Dependency: fungsi yang hasilnya disuntikkan ke endpoint lewat `Depends`. FastAPI memanggilnya untuk setiap request.

`yield`: sesi dibuat sebelum endpoint jalan dan ditutup sesudahnya, berhasil atau gagal.

Pola yang sama dipakai untuk pengguna yang sedang login di Bagian 7. ORM SQLAlchemy dari slide lama dibahas penuh di P12.

Inti Bagian 6

Depends menyuntikkan sesi basis data ke setiap endpoint; tabelnya di P12.

Bagian 7: Keamanan dan autentikasi

TUJUAN: MENJELASKAN AUTENTIKASI DENGAN TOKEN DAN PRAKTIK KEAMANAN DASAR API

Praktik keamanan dasar

Ancaman atau kebutuhan	Praktiknya di FastAPI
password bocor	simpan hash, bukan teks asli
siapa yang memanggil?	login memberi token (JWT); request membawa header `Authorization`
khusus pengguna login	dependency pembaca token; 401 bila tidak sah
berhak atau tidak	periksa peran; 403 bila tidak berhak
data berbahaya	validasi Pydantic; query lewat ORM, bukan SQL rakitan
asal yang tidak dikenal	CORS hanya untuk asal frontend kalian

Inti Bagian 7

Hash password, token di header Authorization, diperiksa lewat dependency.

Bagian 8: API dan RESTful services

TUJUAN: MERANCANG ENDPOINT MENGIKUTI PRINSIP REST DAN MEMANGGILNYA DARI NEXT.JS

Satu sumber daya, lima endpoint

Metode dan path	Tindakan	Berhasil
`GET /matkul`	daftar, dengan query penyaring	200
`GET /matkul/{kode}`	satu mata kuliah	200, atau 404
`POST /matkul`	tambah dari body	201, atau 422
`PUT /matkul/{kode}`	ubah seluruhnya	200, atau 404
`DELETE /matkul/{kode}`	hapus	204, atau 404

Ingat

Path memakai kata benda jamak, bukan kata kerja: `POST /matkul`, bukan `/tambahMatkul`. Metode HTTP sudah menyatakan tindakannya (P1).

Memanggil dari Next.js

```
async function tambah(nilai) {
  const res = await fetch("http://localhost:8000/matkul", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(nilai),
  });
  if (res.status === 422) {
    const { detail } = await res.json();
    setGalat(detail[0].msg);
    return;
  }
  if (!res.ok) { setGalat("Gagal menyimpan. Coba lagi."); return; }
  setPesan("Mata kuliah tersimpan.");
}
```

Sisi client dari endpoint POST tadi, di formulir terkendali P7.

Body dikirim sebagai JSON dengan header `Content-Type`; `JSON.stringify` mengubah object menjadi teks.

Respons 422 berisi `detail`: daftar kolom yang salah beserta pesannya; tampilkan ke pengguna di dekat kolomnya.

Status lain di luar 2xx: pesan gagal. Berhasil: konfirmasi. Tiga keadaan antarmuka dari P1.

Tiga baris yang salah

Yang ditulis	Mengapa salah, dan perbaikannya
`return {"error": "tidak ada"}` untuk data tak ditemukan	Statusnya 200, sehingga frontend mengira berhasil. `raise HTTPException(404, ...)`.
`@app.get("/hapusMatkul/{kode}")`	GET tidak boleh mengubah data dan path memakai kata kerja. `@app.delete("/matkul/{kode}")`.
fetch dari Next.js gagal, Network menunjukkan 200	Diblokir CORS. Tambahkan asal `http://localhost:3000` di `CORSMiddleware`.

Pola kesalahannya

Ketiganya berasal dari kebiasaan lama: mengabaikan kode status dan metode HTTP dari P1.

Giliran kalian

SETENGAH JADI: HAPUS MATA KULIAH

Lengkapi: `@app._____("/matkul/{kode}", status_code=____)` dan di dalam fungsi, bila tidak ditemukan `raise HTTPException(status_code=____)`.

MANDIRI: PENCARIAN

Tulis endpoint `GET /matkul/cari` dengan query `q` wajib minimal 2 huruf yang mengembalikan mata kuliah yang namanya mengandung `q`, tanpa membedakan huruf besar.

JAWABAN

Setengah jadi: delete, 204, 404. Mandiri: def cari(q: Annotated[str, Query(min_length=2)]): return [m for m in DATA if q.lower() in m[”nama”].lower()]; tulis sebelum /matkul/{kode} agar tidak tertangkap sebagai kode.

Inti Bagian 8

Kata benda di URL, tindakan di metode HTTP, kode status yang tepat.

Latihan mandiri

Kerjakan lembar dan berkas awal di folder `02 Hands-on/P11 - FastAPI - Hands-on/`.

Tingkat	Waktu	Yang dilatih
Tingkat 1, Dasar	10’	path operation, path parameter, query parameter, dan 404
Tingkat 2, Menengah	15’	model Pydantic, POST 201, PUT, DELETE 204, dan 422
Tingkat 3, Tantangan	10’	CORS dan menyambungkan Next.js P7 ke API
Pengayaan	sisa	APIRouter dan dependency sederhana

Kesalahan yang sering muncul

Yang ditulis	Masalahnya	Obatnya
200 untuk data tidak ada	frontend mengira berhasil	`HTTPException(404)`
kata kerja di path	melanggar REST	kata benda jamak dan metode HTTP
lupa CORS	browser memblokir response	`CORSMiddleware` untuk asal Next.js
validasi dengan if manual	panjang dan mudah terlewat	`Field` di model Pydantic
route berparameter sebelum route tetap	`/matkul/cari` tertangkap sebagai kode	tulis route tetap lebih dulu
password disimpan apa adanya	bocor bila basis data bocor	simpan hash

Rangkuman

No	Bagian	Inti
1	Pengenalan FastAPI	FastAPI membaca type hints untuk validasi, JSON, dan dokumentasi otomatis.
2	Arsitektur FastAPI	Middleware, path operation, validasi Pydantic, dependency, lalu balasan JSON.
3	Instalasi dan konfigurasi	Pasang di venv, jalankan fastapi dev, izinkan asal Next.js lewat CORS.
4	Routing	Dekorator membuat endpoint; path parameter menunjuk, query parameter menyaring.
5	Request dan response model	Body divalidasi model Pydantic, ditolak 422 bila salah; response model menyaring keluaran.
6	Model dan integrasi basis data	Depends menyuntikkan sesi basis data ke setiap endpoint; tabelnya di P12.
7	Keamanan dan autentikasi	Hash password, token di header Authorization, diperiksa lewat dependency.
8	API dan RESTful services	Kata benda di URL, tindakan di metode HTTP, kode status yang tepat.

Cek pemahaman

Jawab tanpa membuka modul atau slide. Mengingat kembali tanpa melihat lebih menguatkan ingatan daripada membaca ulang.

No	Soal
1	Dari mana FastAPI tahu SKS harus angka?
2	Apa beda path parameter dan query parameter?
3	Kode status bila body tidak lolos validasi Pydantic?
4	Untuk apa `response_model`?
5	Mengapa Next.js di port 3000 butuh CORS untuk memanggil port 8000?
6	Path dan metode untuk menghapus mata kuliah IF25-22014 menurut REST?

KUNCI CEK PEMAHAMAN

(1) Dari type hint int di parameter atau model. (2) Path parameter bagian URL yang menunjuk sumber daya; query parameter setelah tanda tanya untuk penyaring atau opsi. (3) 422. (4) Menyaring dan mendokumentasikan data yang dikirim balik. (5) Asal berbeda; browser memblokir response tanpa izin CORS dari server. (6) DELETE /matkul/IF25-22014, berhasil 204.

Bacaan lanjut

FastAPI, *Tutorial - User Guide*: First Steps, Path Parameters, Query Parameters, Request Body, Response Model, Handling Errors, CORS, Dependencies, dan Security (`fastapi.tiangolo.com/tutorial`). Pydantic, *Fields* (`docs.pydantic.dev`).